Team Name : AjGreat

Team Leader Name : Ajeet Singh

Problem Statement : Intelligent Candidate Discovery & Ranking Challenge.

Solution Overview

- Built an offline two-stage ranker where rubric.yaml defines the ideal Senior AI Engineer profile.
- Candidates are retrieved and reranked by comparing resumes against that rubric using structured signals.

JD Understanding & Candidate Evaluation

- The rubric captures senior AI/ML search experience, production retrieval/ranking systems, Python, evaluation metrics, and product-company fit.
- Candidate relevance is judged beyond keywords using career evidence, trusted skills, India logistics, Redrob behavior, and availability.

Ranking Methodology

- Stage 1 streams 100K candidates and keeps the strongest 1500 using rubric-derived role, skill, evidence, location, company, and behavior signals.
- Stage 2 applies capped scoring plus honeypot penalties to produce the final top 100 sorted candidates.

Explainability & Data Validation

- Reasoning is generated only from candidate JSON facts such as title, years, company, career evidence, location, and Redrob signals.
- Suspicious profiles are downranked for keyword stuffing, expert skills with zero duration, inconsistent experience, weak evidence, or poor availability.

End-to-End Workflow

- Workflow: read JD rubric, stream candidates, score against the ideal profile, retrieve top 1500, rerank top 100, and write CSV.
- The output is validated with validate_submission.py and audited with ranking_audit.json for ordering, reasoning, and risk flags.

System Architecture

Architecture: rubric.yaml + candidates.jsonl feed rank.py scoring modules for role, skills, evidence, company, location, behavior, and penalties.

A stage-1 heap retrieval feeds the stage-2 reranker, which writes submission.csv and artifacts/ranking_audit.json.

Results & Performance

- The full 100K-candidate ranking runs offline on CPU in about 16 seconds and produces 100 unique grounded explanations.
- No network, hosted API, GPU, local LLM, or live embedding model is used during ranking.

Technologies Used

- Ranking uses Python stdlib only for reproducible CPU execution; pandas is used separately for offline feature profiling.
- The design avoids runtime ML dependencies while still supporting optional precomputed dense similarity artifacts.

Submission Assets

Submission assets include GitHub repo, team_AjGreat.csv/submission.csv, README reproduction steps, metadata YAML, and audit report.

Feature profiles and suggested feature sets document how company, industry, title, and skill signals were derived.

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU